

Laboratorio #3

Clasificación de texto con Naive Bayes

Instrucciones generales

- Individual.
- Entrega: Lunes 17 de agosto, 2026. 23:59.
- Trabajar en Python, en un Jupyter Notebook o script (.py).

Antes de iniciar, asegúrese de tener disponibles en su notebook, desde los laboratorios anteriores:

- El DataFrame del corpus con el texto original, el texto normalizado y la columna de categoría.
- El pipeline de normalización completo.
- Las representaciones vectoriales del corpus.

1. Conjuntos de entrenamiento, validación y prueba

Antes de entrenar cualquier modelo, separe el corpus siguiendo la lógica de aprendizaje supervisado vista en clase: el modelo debe aprender de un conjunto de entrenamiento y su desempeño debe medirse sobre datos que nunca vio.

- Divida el corpus normalizado en conjuntos de entrenamiento (70%), validación (15%) y prueba (15%), usando `train_test_split` de `scikit-learn`.
- Utilice estratificación por categoría (`stratify`) para que la proporción de clases se mantenga similar en los tres conjuntos.
- Reporte el número de documentos y la distribución de categorías en cada conjunto (tabla o gráfico de barras).
- Explique con sus propias palabras para qué sirve cada conjunto (entrenamiento, validación, prueba) y por qué no se debe usar el conjunto de prueba para tomar decisiones durante el desarrollo del modelo.

2. Construcción del clasificador Naive Bayes

Entrene un clasificador Naive Bayes Multinomial (`MultinomialNB` de `scikit-learn`), apropiado para datos de conteo como los que produce una Bolsa de Palabras o TF-IDF.

- Entrene un primer modelo usando como features la matriz de Bolsa de Palabras (CountVectorizer) del Laboratorio #2, ajustada únicamente con el conjunto de entrenamiento.
- Entrene un segundo modelo usando como features la matriz TF-IDF (TfidfVectorizer), ajustada de la misma forma.
- **Importante:** ajuste (fit) los vectorizadores solo con el conjunto de entrenamiento y luego transforme (transform) validación y prueba con ese mismo vocabulario, para evitar fuga de información.
- Explique, con sus propias palabras y apoyándose en el teorema de Bayes visto en clase, qué representan $P(c)$ y $P(w_i|c)$ en el contexto de este corpus de noticias, y cómo el modelo combina ambas probabilidades para clasificar un documento nuevo.

3. Entrenamiento y evaluación

Evalúe ambos modelos (BoW y TF-IDF) sobre el conjunto de validación y, al final, sobre el conjunto de prueba.

- Reporte accuracy, precision, recall y F1-score (macro y ponderado) para cada modelo, usando classification_report de scikit-learn.
- Construya y grafique la matriz de confusión de cada modelo (por ejemplo con seaborn.heatmap), con las categorías del corpus en los ejes.
- Compare el desempeño en entrenamiento contra el desempeño en validación/prueba. ¿Hay indicios de overfitting? Relacione su respuesta con lo visto en clase sobre generalización.
- Indique cuál de los dos modelos (BoW o TF-IDF) obtuvo mejor desempeño y proponga una explicación, en términos de cómo cada representación pondera las palabras.

4. Análisis de errores

Con base en la matriz de confusión y en el classification_report del mejor modelo:

- Identifique las dos categorías que el modelo confunde con mayor frecuencia entre sí. Revise 3 a 5 documentos mal clasificados de esas categorías y comente qué palabras pudieron llevar al modelo a equivocarse.
- Calcule y liste las palabras con mayor $P(w_i|c)$ (verosimilitud) para dos o tres categorías del corpus. ¿Tienen sentido como palabras representativas de esa categoría?

- Relacione los errores encontrados con el supuesto de independencia condicional de Naive Bayes: ¿en qué casos cree que este supuesto "ingenuo" contribuyó a una clasificación incorrecta?

5. Efecto de la representación de texto

Explore cómo cambian los resultados al modificar la representación usada como entrada del clasificador.

- Repita el entrenamiento y evaluación del modelo BoW limitando el vocabulario a las 1000 y 5000 palabras más frecuentes (max_features). Compare el desempeño contra el vocabulario completo.
- Repita el entrenamiento usando bigramas (ngram_range=(1,2)) además de unigramas. ¿Mejora el desempeño respecto al modelo de solo unigramas? ¿A qué costo, en tamaño de vocabulario y tiempo de entrenamiento?
- Resuma en una tabla comparativa (representación, tamaño de vocabulario, accuracy, F1 macro) todos los modelos entrenados en este laboratorio, incluyendo su implementación propia.

6. Análisis

Apoyándose en los resultados obtenidos, responda:

1. ¿Por qué Naive Bayes es un clasificador probabilístico y no uno basado en distancias o similitud, como los usados en el Laboratorio #2? ¿Qué ventajas y limitaciones le encuentra frente a comparar documentos por similitud coseno?
2. El supuesto de independencia condicional entre palabras es, en principio, incorrecto para el lenguaje natural. A partir de sus resultados, ¿por qué cree que el modelo funcionó razonablemente bien de todas formas? Relacione su respuesta con las razones vistas en clase (señales débiles que se suman, robustez con datos dispersos, importancia del orden relativo de los scores).
3. Al implementar Naive Bayes desde cero, ¿qué parte del algoritmo le resultó más difícil de traducir a código (el cálculo de $P(c)$, la verosimilitud $P(w_i|c)$, el suavizado o el uso de logaritmos)? ¿Qué aprendió sobre el modelo al construirlo usted mismo que no era evidente al usar MultinomialNB directamente?
4. ¿Qué categorías del corpus fueron más fáciles de clasificar y cuáles más difíciles? ¿Coincide con las categorías que ya se confundían al usar similitud coseno en el Laboratorio #2?

5. Compare el uso de un conjunto de validación separado del de prueba: ¿en qué momento de este laboratorio usó cada uno y por qué es importante no mezclarlos?
6. Si tuviera que poner este clasificador en producción para etiquetar noticias nuevas automáticamente, ¿qué riesgos le preocuparían (por ejemplo, categorías nuevas no vistas en entrenamiento, cambios en el vocabulario con el tiempo, clases desbalanceadas)? ¿Cómo los mitigaría?

Entregables

Entregable	Contenido
Link a repositorio	Repositorio (por ejemplo GitHub) con el código versionado del laboratorio.
Reporte escrito (máx. 4 páginas)	Resultados de las secciones 1 a 5 (particiones, modelos, métricas, matrices de confusión, implementación propia de Naive Bayes y su comparación con scikit-learn, análisis de errores y comparación de representaciones) y respuestas al análisis (sección 7).